
NOTE

While the following is an implementation detail, it may help to further your understanding of objects. Internally, all instance variables of an object are kept as name/value pairs in a Python dictionary. You can inspect all the instance variables in an object by calling the built-in `vars()` function on any object. For example, in the test code from [Listing 3-2](#), if you want to see the internal representation of the instance variables, you can add this line at the end:

```
print('oDimmer1 variables:', vars(oDimmer1))
```

When you run it, you'll see this output:

```
oDimmer1 variables: {'label': 'Dimmer1', 'isOn': True, 'brightness': 2}
```

What Is the Meaning of “self”?

Philosophers have struggled with this question for centuries, so it would be rather pretentious of me to try to explain it in just a few pages. In Python, however, the variable named `self` does have a highly specialized and clear meaning. In this section, I'll show how `self` is given a value and how the code of the methods in a class work with the instance variables of any object instantiated from the class.

NOTE

The variable name `self` is not a keyword in Python but is used by convention—any other name could be used and the code would work fine. However, using `self` is a universally accepted practice in Python, and I will use it throughout this book. If you want your code to be understood by other Python programmers, use the name `self` as the first parameter in all methods of a class. (Other OOP languages have the same concept but use other names, such as `this` or `me`.)
